Ex 3: Generative Adversarial Networks (GANs) for Synthetic Image Generation

Learning Objective:
Implementation of Generative Adversarial Networks (GANs) for Synthetic Image Generation using the MNIST Dataset.

STEPS :

1. Load MNIST dataset, normalize images to range -1,1. Split the Batch and shuffle the dataset.
2. Build the Generator
    a. Input: random noise vector (100-dim).
    a. Apply Dense → reshape to $7\times7\times256$.
    b. Apply BatchNorm + LeakyReLU.
    c. Upsample using Conv2DTranspose layers to $28\times28\times1$.
    d. Output activation: tanh.
2. Build the Discriminator
    a. Input: $28\times28\times1$ image.
    a. Apply Conv2D + LeakyReLU + Dropout layers.
    b. Flatten output.
    c. Final Dense layer with Sigmoid → real or fake probability.
2. Define Loss Functions
    a. Discriminator:
        . Real images → label 1
        i. Fake images → label 0
    a. Generator:
        . Fake images should be classified as real (label 1)
    a. Use Binary Cross-Entropy loss.
2. Define Adam optimizer for both networks.
3. Training Step
    a. Generate random noise.
    a. Produce fake images using Generator.
    b. Evaluate real and fake images using Discriminator.
    c. Compute Generator and Discriminator losses.
    d. Compute gradients and update weights.
2. Training Loop
    a. Repeat training step for each batch in each epoch.
    a. Periodically generate and display sample images.
2. Visualize Output by comparing generated fake images with the original images.

PROGRAM:

```
# Import Libraries
import tensorflow as tf
from tensorflow.keras import layers
import matplotlib.pyplot as plt
import numpy as np
```

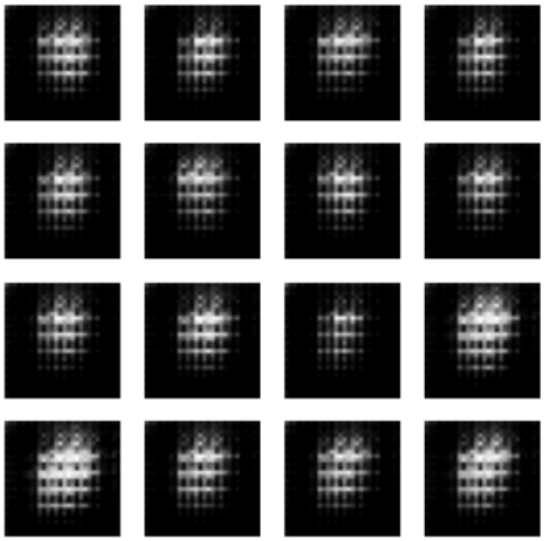
```
# Load MNIST Dataset
(train_images, train_labels), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize images to [-1,1]
train_images = train_images.reshape(train_images.shape[0], 28, 28, 1).astype('float32')
train_images = (train_images - 127.5) / 127.5

# Batch and shuffle
BUFFER_SIZE = 60000
BATCH_SIZE = 256

train_dataset =
tf.data.Dataset.from_tensor_slices(train_images).shuffle(BUFFER_SIZE).batch(BATCH_SIZE)

# Generator Model
def build_generator():
    model = tf.keras.Sequential()

    model.add(layers.Dense(7*7*256, use_bias=False, input_shape=(100,)))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Reshape((7,7,256)))

    model.add(layers.Conv2DTranspose(128,(5,5),strides=(1,1),padding='same',use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())

    model.add(layers.Conv2DTranspose(64,(5,5),strides=(2,2),padding='same',use_bias=False))
    model.add(layers.BatchNormalization())
    model.add(layers.LeakyReLU())


model.add(layers.Conv2DTranspose(1,(5,5),strides=(2,2),padding='same',use_bias=False,activati
on='tanh'))

    return model

generator = build_generator()

# Discriminator Model
def build_discriminator():
    model = tf.keras.Sequential()

    model.add(layers.Conv2D(64,(5,5),strides=(2,2),padding='same',input_shape=[28,28,1]))
```

```
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Conv2D(128,(5,5),strides=(2,2),padding='same'))
    model.add(layers.LeakyReLU())
    model.add(layers.Dropout(0.3))

    model.add(layers.Flatten())
    model.add(layers.Dense(1,activation='sigmoid'))

    return model

discriminator = build_discriminator()

# Loss Function
cross_entropy = tf.keras.losses.BinaryCrossentropy()

def discriminator_loss(real_output, fake_output):
    real_loss = cross_entropy(tf.ones_like(real_output), real_output)
    fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
    return real_loss + fake_loss

def generator_loss(fake_output):
    return cross_entropy(tf.ones_like(fake_output), fake_output)

# Optimizers
generator_optimizer = tf.keras.optimizers.Adam(1e-4)
discriminator_optimizer = tf.keras.optimizers.Adam(1e-4)

# Training Step
noise_dim = 100

@tf.function
def train_step(images):

    noise = tf.random.normal([BATCH_SIZE, noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:

        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = generator_loss(fake_output)
        disc_loss = discriminator_loss(real_output, fake_output)
```

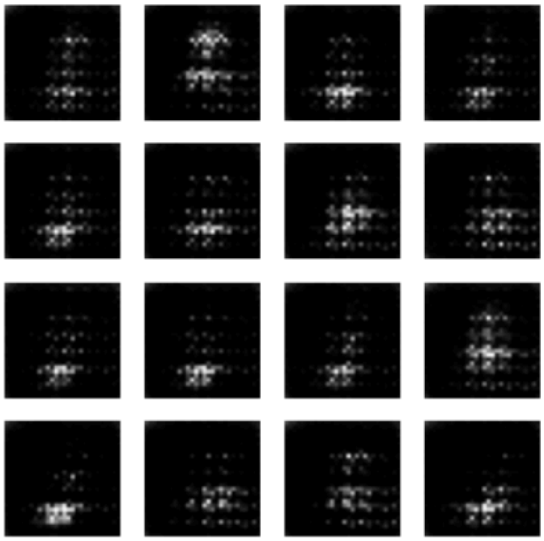
```
    gradients_gen = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_disc = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_gen, generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_disc,
discriminator.trainable_variables))

# Generate and display images
def generate_images(model, epoch):

    noise = tf.random.normal([16,100])
    predictions = model(noise, training=False)

    fig = plt.figure(figsize=(4,4))

    for i in range(predictions.shape[0]):
        plt.subplot(4,4,i+1)
        plt.imshow(predictions[i,:,:,0]*127.5 + 127.5, cmap='gray')
        plt.axis('off')

    plt.show()

# Training Loop
EPOCHS = 10

for epoch in range(EPOCHS):
    for image_batch in train_dataset:
        train_step(image_batch)

    print("Epoch:",epoch+1)
    generate_images(generator,epoch)

# Compare real images
plt.figure(figsize=(4,4))
for i in range(16):
    plt.subplot(4,4,i+1)
    plt.imshow(train_images[i,:,:,0]*127.5+127.5,cmap='gray')
    plt.axis('off')
plt.show()
```

OUTPUT:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserW
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch: 1
```

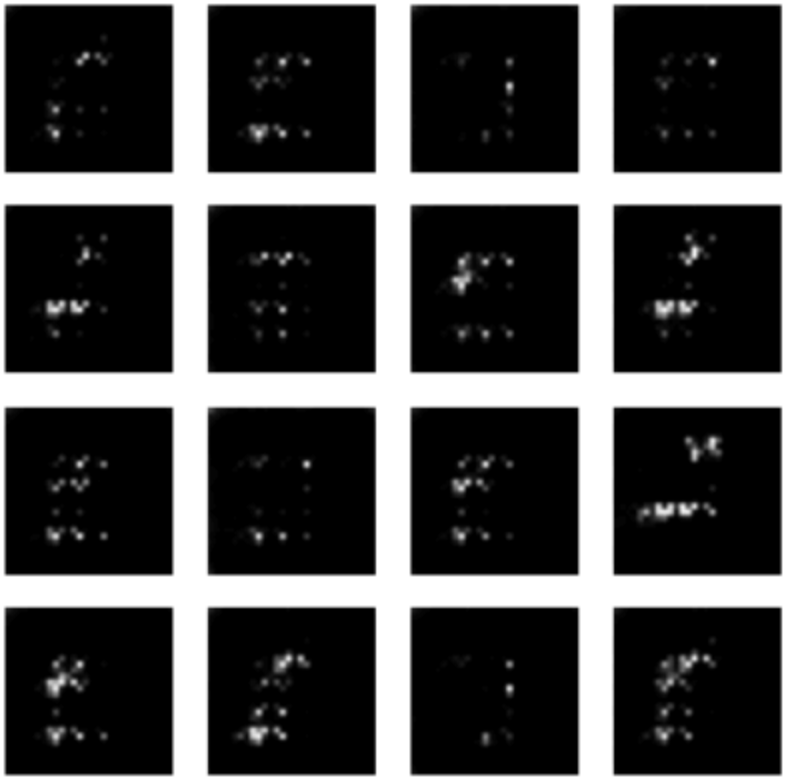


```
Epoch: 2
```

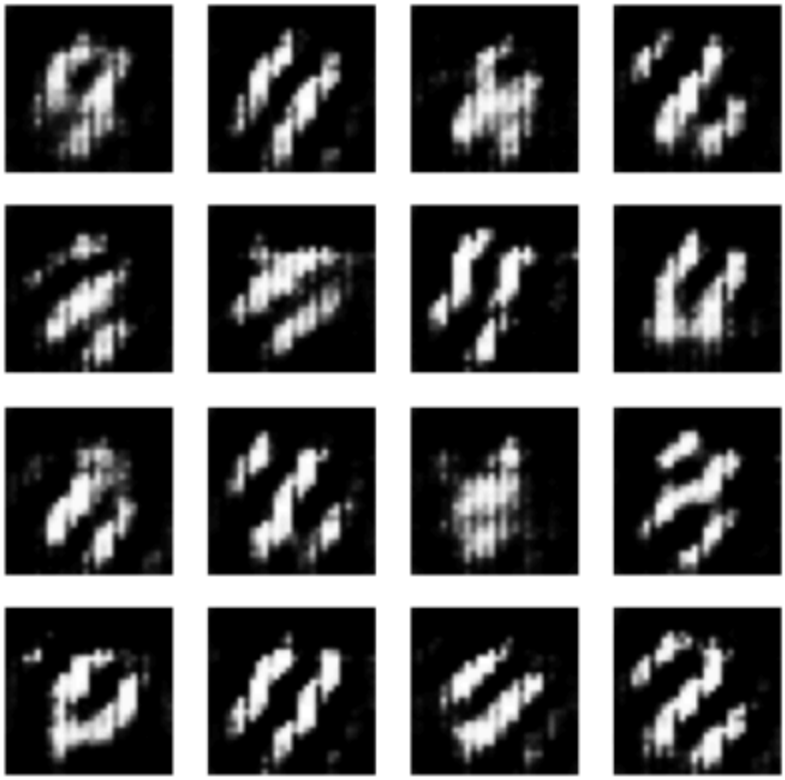


```
Epoch: 3
```

Epoch: 3

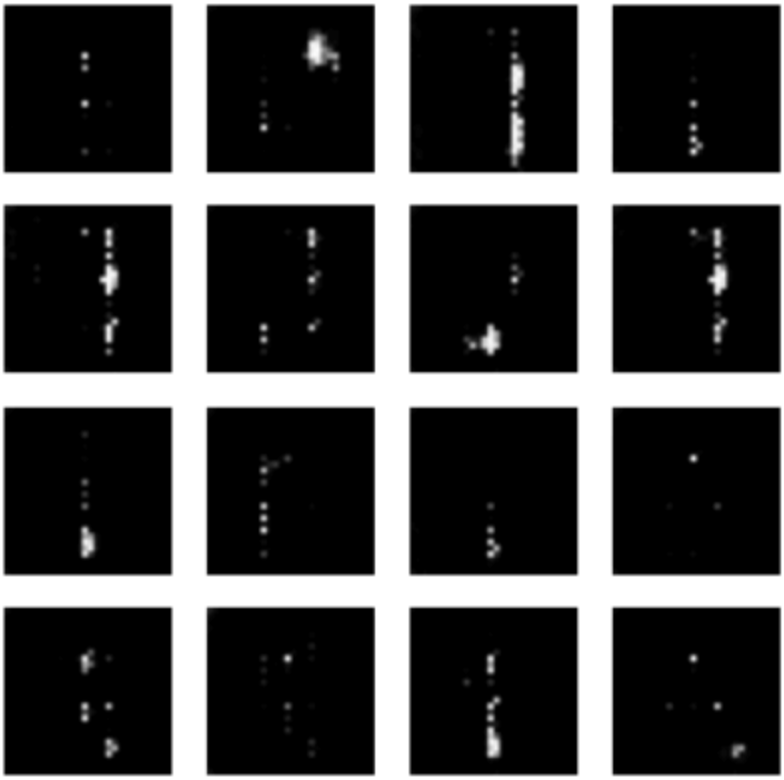


Epoch: 4



Epoch: 5

Epoch: 5



RESULT :

Hence, the implementation of Generative Adversarial Networks for synthetic image generation was successfully completed.